

Outage-Calibrated Robust Design for Active STAR-RIS Assisted URLLC IoT Networks Under Imperfect CSI

Research Progress by
Amudhavan M



INTRODUCTION

URLLC deadlines are hard deadlines :

Ultra-reliable low-latency communication targets millisecond-scale deadlines for IoT services such as industrial automation. Missing the deadline is a failure, not a slowdown.

Obstacles break the direct link :

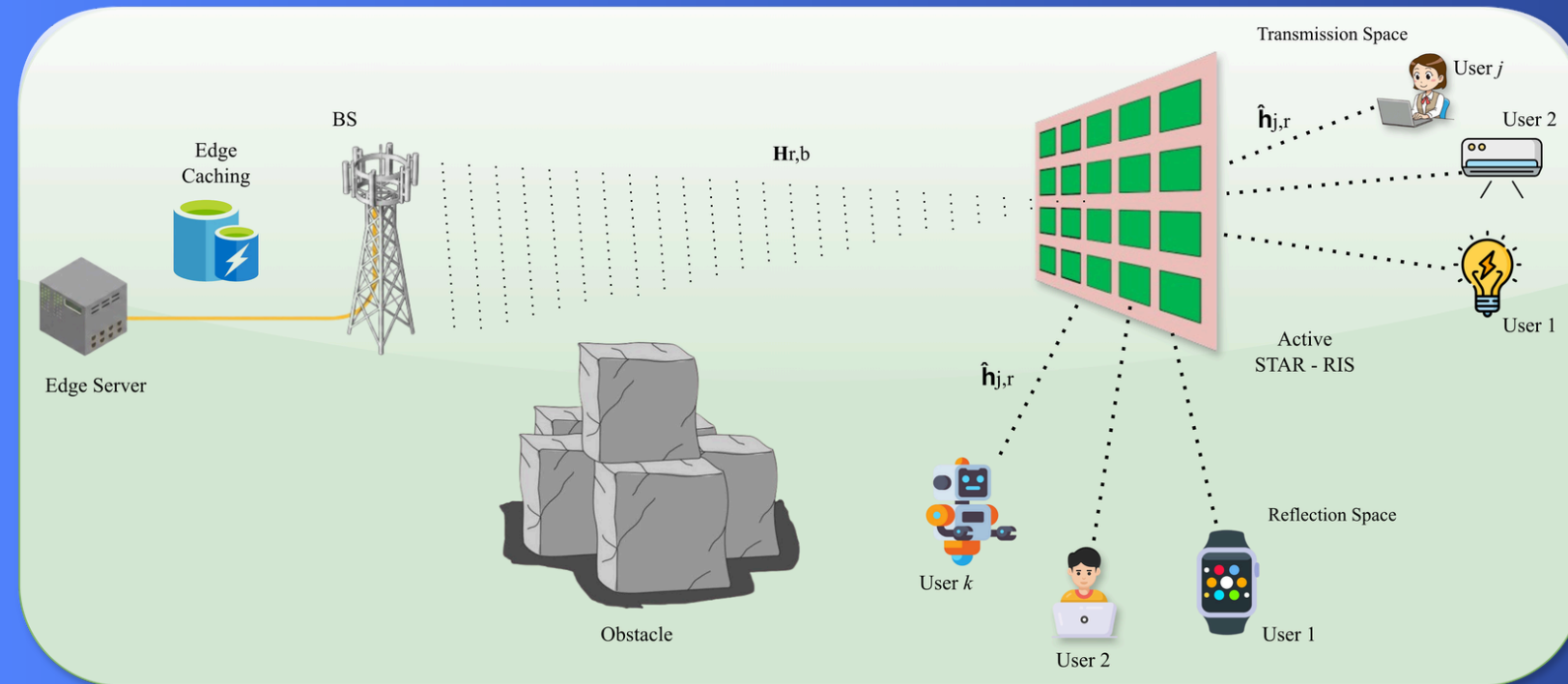
Blockage severs the IoT device to base station path. An active STAR-RIS restores connectivity to users on both sides of the surface - transmission and reflection which a passive, reflectiononly RIS cannot do.

The gap in prior work

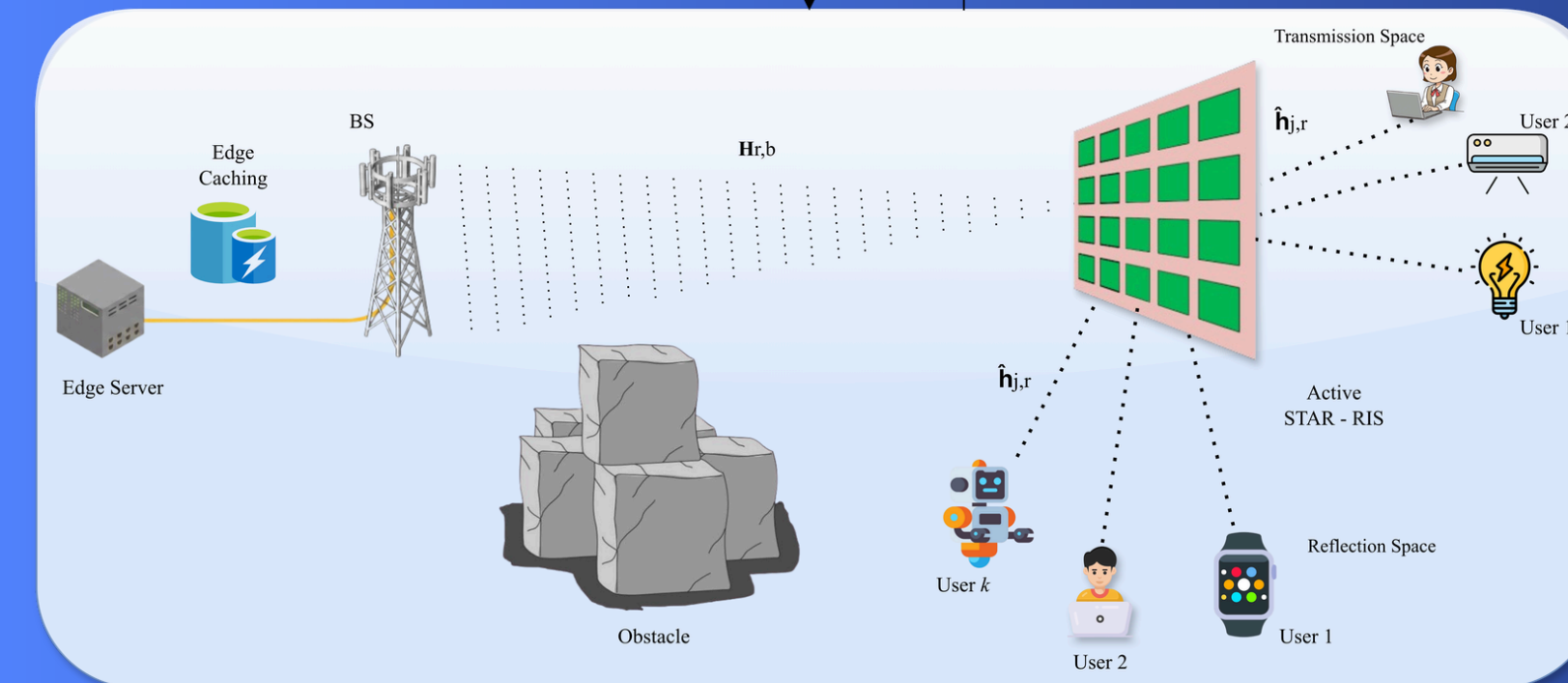
The gap in prior work Lestari et al. (ICC 2024) optimize assuming the channel is known exactly. Real channel knowledge is always somewhat wrong.

System Model

Physical World



Real Time



Virtual World

- Single-antenna IoT nodes, split into reflection-side users J_r and transmission-side users J_t
- Antennas at the base station
- Each of the RIS's tunable elements has two knobs, how strong it makes the signal and how it shifts its timing.

$$\gamma_j = |\mathbf{w}_j^H \mathbf{h}_j|^2 p_j / (\|\mathbf{w}_j^H \mathbf{H}_{rb} \Psi\|^2 \sigma_0^2 + \|\mathbf{w}_j\|^2 \mathbf{B} \mathbf{b}_j \mathbf{N}_0)$$

Problem Formulation

Objective:

Minimize total latency across all J users, over transmit power, bandwidth allocation, BS beamforming, and the active STAR-RIS coefficients.

Chance constraints are required because the channel is uncertain

$$\Pr\{ T_j^{\text{tot}} \leq T_{\text{max}} \} \geq 1 - p_{\text{out}}$$

deadline is met with the required confidence

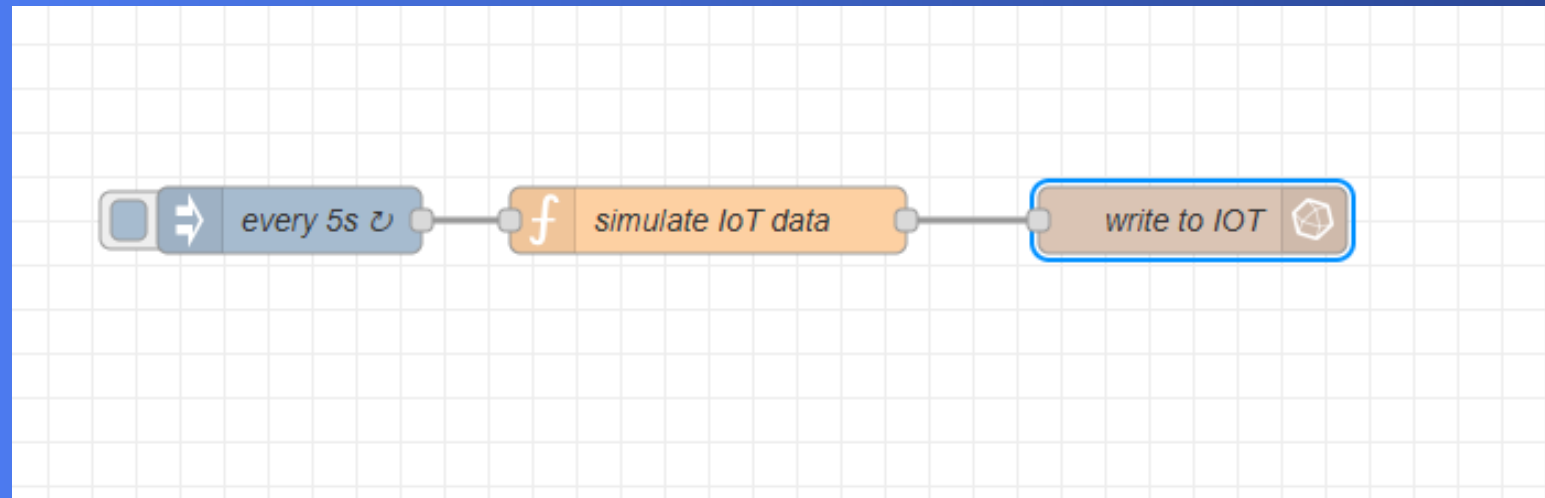
$$\Pr\{ R_j \geq R_{\text{min}} \} \geq 1 - p_{\text{out}}$$

finite-blocklength rate floor holds with the same confidence

Also enforced: per-node energy budget, total bandwidth, edge caching capacity, and RIS hardware limits on per-element amplitude and phase.

Solved by AO over the 5-variable blocks.

IOT Networks



```
1 msg.payload = {  
2   node_id: Math.floor(Math.random()*15),  
3   rssi: -60 - Math.random()*30,  
4   x: 40 + Math.random()*20,  
5   y: -20 + Math.random()*40  
6 };  
7 return msg;
```

1. Used Node-Red to simulate the IOT data.
2. InfluxDB for storing the IOT Data.
3. Received Signal Strength Indicator, in dBm (-60 is stronger) (-90 is weaker)
4. X and Y are IOT device positions.
5. For complex channel data (phase + amplitude), used numpy's Rician fading model instead, since no sensor/database can capture phase.

Imperfect CSI Model

The optimizer never sees the true channel — only an estimate, plus a bounded error.

$$\mathbf{h}_{r,j} = \hat{\mathbf{h}}_{r,j} + \mathbf{e}_j, \quad \|\mathbf{e}_j\| \leq \kappa \|\hat{\mathbf{h}}_{r,j}\|$$

- $\mathbf{h}$ - the true channel (unknown)
- $\hat{\mathbf{h}}$ - the estimate (what we use)
- $\mathbf{e}$ - the error, bounded by κ
- κ - CSI error level, swept 0–20%

Only the user to RIS link is uncertain. The RIS to BS link is known.

How I Solved the Problem: A Calibrated Margin

The classical fix - worst case:

Assume the error always points in the single most damaging direction, then design against that. With $N = 32$ elements, that exact alignment is vanishingly unlikely the margin ends up enormous and almost never needed.

My idea: calibrate the margin instead

Size the margin to the actual probability of damage - not the worst case.

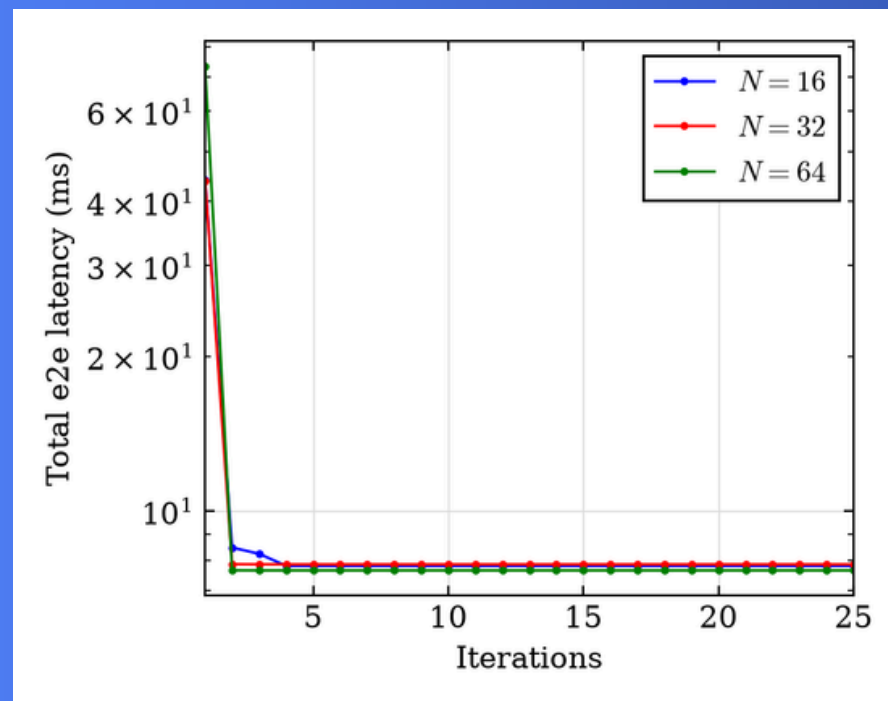
$$t^* = \sqrt{(1 - p_{\text{out}}^{1/(N-1)})}$$

At $N = 32$, $p_{\text{out}} = 1\%$: $t^* \approx 0.37$ - about one third of the worst-case margin.

Results

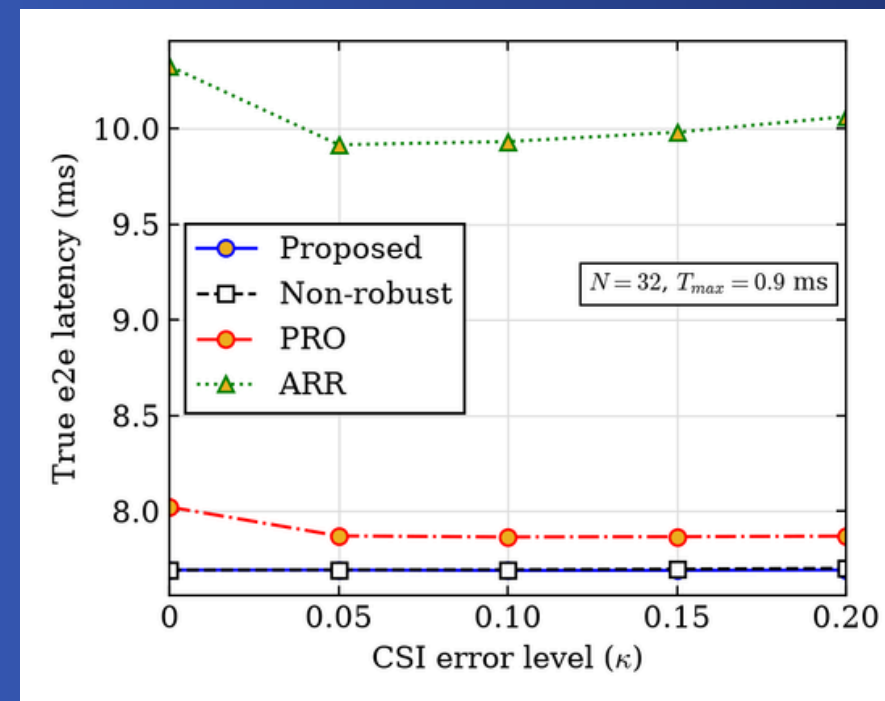
Convergence:

The algorithm converges in just a few iterations; regardless of how many RIS elements we use, it's not sensitive to the random starting point.



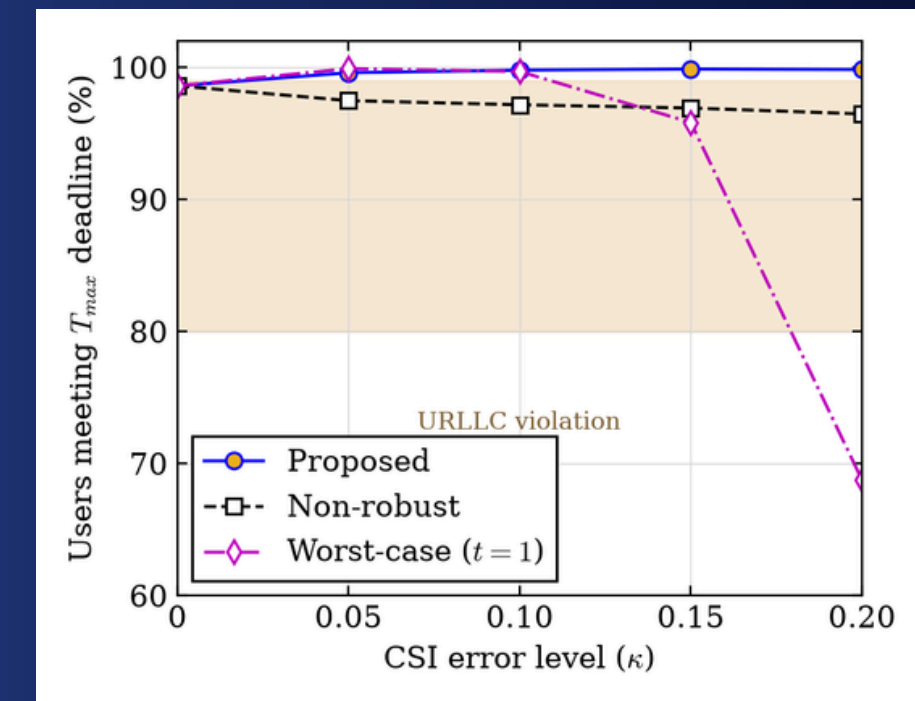
Latency vs CSI error:

Our proposed design's latency barely changes as the CSI error grows the robustness costs almost nothing in speed. ARR (random-phase) is clearly the slowest scheme throughout.



URRLLC :

Calibration beats both ignoring the error and over-protecting against it the worst-case margin actually collapses below the non-robust design at high error.



Conclusion

Robustness built in, not bolted on:

The CSI error is carried inside the optimizer itself, not just measured afterward.

Calibrated, not worst-case:

sizing the margin to the true failure probability beats both ignoring the error and over-protecting against it.

Thank you!